

From the above discussion we have seen that, if we select median of medians m as pivot, the partition sizes are: $\frac{3n}{10} - 6$ and $\frac{7n}{10} + 6$. If we select the maximum of these, then we get:

$$\begin{aligned}
 T(n) &= T\left(\frac{n}{5}\right) + \Theta(n) + T\left(\frac{7n}{10} + 6\right) \\
 &\approx T\left(\frac{n}{5}\right) + \Theta(n) + T\left(\frac{7n}{10}\right) + O(1) \\
 &\leq c \frac{7n}{10} + c \frac{n}{5} + \Theta(n) + O(1)
 \end{aligned}$$

Finally, $T(n) = \Theta(n)$.

Problem-12 In [Problem-11](#), we divided the input array into groups of 5 elements. The constant 5 play an important part in the analysis. Can we divide in groups of 3 which work in linear time?

Solution: In this case the modification causes the routine to take more than linear time. In the worst case, at least half of the $\lceil \frac{n}{3} \rceil$ medians found in the grouping step are greater than the median of medians m , but two of those groups contribute less than two elements larger than m . So as an upper bound, the number of elements larger than the pivotpoint is at least:

$$2(\lceil \frac{1}{2} \lceil \frac{n}{3} \rceil \rceil - 2) \geq \frac{n}{3} - 4$$

Likewise this is a lower bound. Thus up to $n - (\frac{n}{3} - 4) = \frac{2n}{3} + 4$ elements are fed into the recursive call to *Select*. The recursive step that finds the median of medians runs on a problem of size $\lceil \frac{n}{3} \rceil$, and consequently the time recurrence is:

$$T(n) = T(\lceil \frac{n}{3} \rceil) + T(2n/3 + 4) + \Theta(n).$$

Assuming that $T(n)$ is monotonically increasing, we may conclude that $T(\frac{2n}{3} + 4) \geq T(\frac{2n}{3}) \geq 2T(\frac{n}{3})$, and we can say the upper bound for this as $T(n) \geq 3T(\frac{n}{3}) + \Theta(n)$, which is $O(n \log n)$. Therefore, we cannot select 3 as the group size.

Problem-13 As in [Problem-12](#), can we use groups of size 7?

Solution: Following a similar reasoning, we once more modify the routine, now using groups of 7 instead of 5. In the worst case, at least half the $\lceil \frac{n}{7} \rceil$ medians found in the grouping step are greater than the median of medians m , but two of those groups contribute less than four elements larger than m . So as an upper bound, the number of elements larger than the pivotpoint is at least:

$$4(\lceil 1/2 \lceil n/7 \rceil \rceil - 2) \geq \frac{2n}{7} - 8.$$

Likewise this is a lower bound. Thus up to $n - (\frac{2n}{7} - 8) = \frac{5n}{7} + 8$ elements are fed into the recursive call to Select. The recursive step that finds the median of medians runs on a problem of size $\lceil \frac{n}{7} \rceil$, and consequently the time recurrence is

$$\begin{aligned}
 T(n) &= T(\lceil \frac{n}{7} \rceil) + T(\frac{5n}{7} + 8) + O(n) \\
 T(n) &\leq c\lceil \frac{n}{7} \rceil + c(\frac{5n}{7} + 8) + O(n) \\
 &\leq c\frac{n}{7} + c\frac{5n}{7} + 8c + an, a \text{ is a constant} \\
 &= cn - c\frac{n}{7} + an + 9c \\
 &= (a + c)n - (c\frac{n}{7} - 9c).
 \end{aligned}$$

This is bounded above by $(a + c)n$ provided that $c\frac{n}{7} - 9c \geq 0$. Therefore, we can select 7 as the group size.

Problem-14 Given two arrays each containing n sorted elements, give an $O(\log n)$ -time algorithm to find the median of all $2n$ elements.

Solution: The simple solution to this problem is to merge the two lists and then take the average of the middle two elements (note the union always contains an even number of values). But, the merge would be $\Theta(n)$, so that doesn't satisfy the problem statement. To get $\log n$ complexity, let *medianA* and *medianB* be the medians of the respective lists (which can be easily found since both lists are sorted). If *medianA* == *medianB*, then that is the overall median of the union and we are done. Otherwise, the median of the union must be between *medianA* and *medianB*. Suppose that *medianA* < *medianB* (the opposite case is entirely similar). Then we need to find the median of the union of the following two sets:

$$\{x \text{ in } A \mid x \geq \text{medianA}\} \cup \{x \text{ in } B \mid x \leq \text{medianB}\}$$

So, we can do this recursively by resetting the *boundaries* of the two arrays. The algorithm tracks both arrays (which are sorted) using two indices. These indices are used to access and compare the median of both arrays to find where the overall median lies.

```

void FindMedian(int A[], int alo, int ahi, int B[], int blo, int bhi) {
    amid = alo + (ahi-alo)/2;
    amed = a[amid];
    bmid = blo + (bhi-blo)/2;
    bmed = b[bmid];
    if( ahi - alo + bhi - blo < 4) {
        Handle the boundary cases and solve it smaller problem in O(1) time.
        return;
    }
    else if(amed < bmed)
        FindMedian(A, amid, ahi, B, blo, bmid+1);
    else
        FindMedian(A, alo, amid+1, B, bmid+1, bhi);
}

```

Time Complexity: $O(\log n)$, since we are reducing the problem size by half every time.

Problem-15 Let A and B be two sorted arrays of n elements each. We can easily find the k^{th} smallest element in A in $O(1)$ time by just outputting $A[k]$. Similarly, we can easily find the k^{th} smallest element in B . Give an $O(\log k)$ time algorithm to find the k^{th} smallest element overall {i.e., the k^{th} smallest in the union of A and B .

Solution: It's just another way of asking [Problem-14](#).

Problem-16 **Find the k smallest elements in sorted order:** Given a set of n elements from a totally-ordered domain, find the k smallest elements, and list them in sorted order. Analyze the worst-case running time of the best implementation of the approach.

Solution: Sort the numbers, and list the k smallest.

$T(n)$ = Time complexity of sort + listing k smallest elements = $\Theta(n \log n) + \Theta(n) = \Theta(n \log n)$.

Problem-17 For [Problem-16](#), if we follow the approach below, then what is the complexity?

Solution: Using the priority queue data structure from heap sort, construct a min-heap over the set, and perform extract-min k times. Refer to the [Priority Queues \(Heaps\)](#) chapter for more details.

Problem-18 For [Problem-16](#), if we follow the approach below then what is the complexity?

Find the k^{th} -smallest element of the set, partition around this pivot element, and sort the k smallest elements.

Solution:

$T(n)$ = Time complexity of k^{th} – smallest + Finding pivot + Sorting prefix
 $= \Theta(n) + \Theta(n) + \Theta(k \log k) = \Theta(n + k \log k)$

Since, $k \leq n$, this approach is better than [Problem-16](#) and [Problem-17](#).

Problem-19 Find k nearest neighbors to the median of n distinct numbers in $O(n)$ time.

Solution: Let us assume that the array elements are sorted. Now find the median of n numbers and call its index as X (since array is sorted, median will be at $\frac{n}{2}$ location). All we need to do is select k elements with the smallest absolute differences from the median, moving from $X - 1$ to 0, and $X + 1$ to $n - 1$ when the median is at index m .

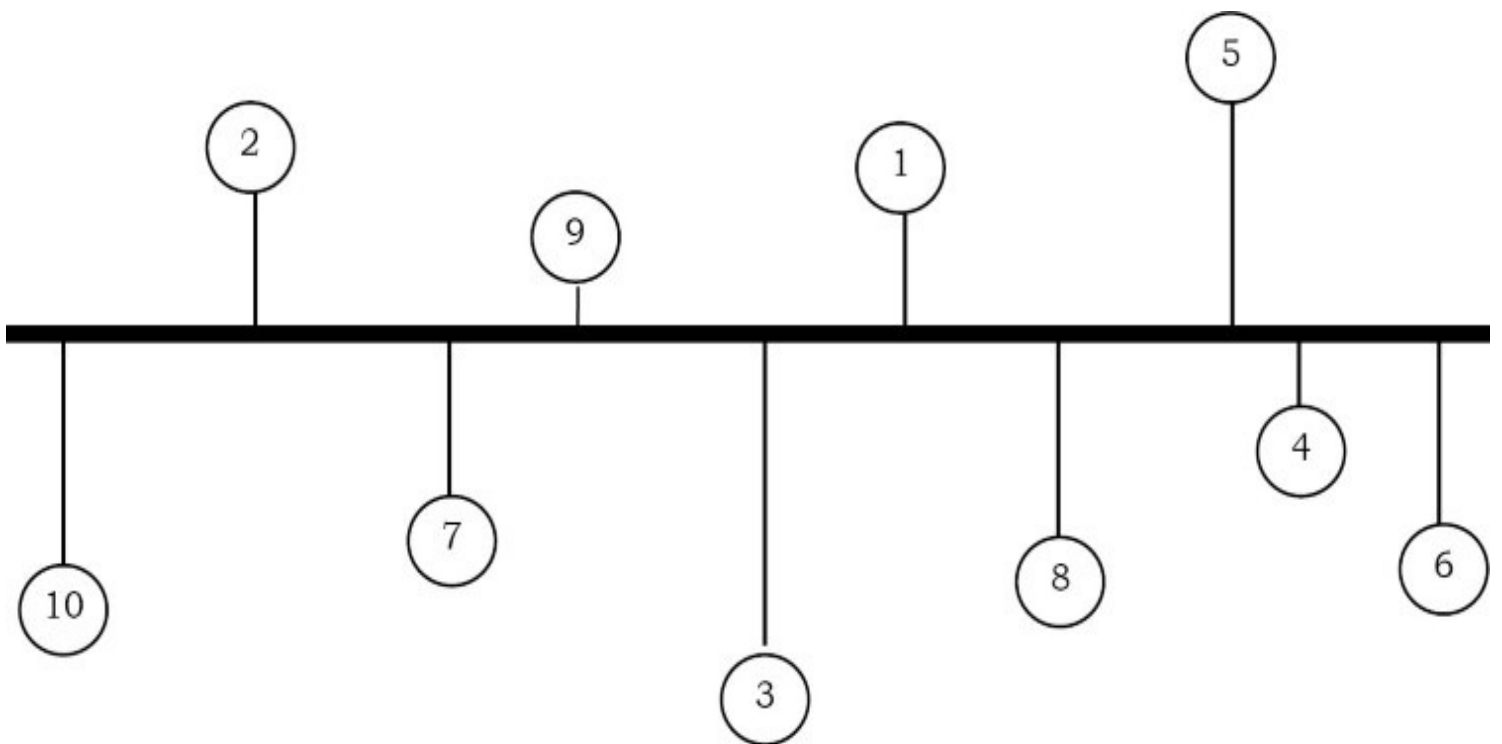
Time Complexity: Each step takes $\Theta(n)$. So the total time complexity of the algorithm is $\Theta(n)$.

Problem-20 Is there any other way of solving [Problem-19](#)?

Solution: Assume for simplicity that n is odd and k is even. If set A is in sorted order, the median is in position $n/2$ and the k numbers in A that are closest to the median are in positions $(n - k)/2$ through $(n + k)/2$.

We first use linear time selection to find the $(n - k)/2$, $n/2$, and $(n + k)/2$ elements and then pass through set A to find the numbers less than the $(n + k)/2$ element, greater than the $(n - k)/2$ element, and not equal to the $n/2$ element. The algorithm takes $O(n)$ time as we use linear time selection exactly three times and traverse the n numbers in A once.

Problem-21 Given (x,y) coordinates of n houses, where should you build a road parallel to x -axis to minimize the construction cost of building driveways?



Solution: The road costs nothing to build. It is the driveways that cost money. The driveway cost is proportional to its distance from the road. Obviously, they will be perpendicular. The solution is to put the street at the median of the y coordinates.

Problem-22 Given a big file containing billions of numbers, find the maximum 10 numbers from that file.

Solution: Refer to the *Priority Queues* chapter.

Problem-23 Suppose there is a milk company. The company collects milk every day from all its agents. The agents are located at different places. To collect the milk, what is the best place to start so that the least amount of total distance is travelled?

Solution: Starting at the median reduces the total distance travelled because it is the place which is at the center of all the places.

SYMBOL TABLES

CHAPTER

13



13.1 Introduction

Since childhood, we all have used a dictionary, and many of us have a word processor (say, Microsoft Word) which comes with a spell checker. The spell checker is also a dictionary but limited in scope. There are many real time examples for dictionaries and a few of them are:

- Spell checker
- The data dictionary found in database management applications
- Symbol tables generated by loaders, assemblers, and compilers
- Routing tables in networking components (DNS lookup)

In computer science, we generally use the term ‘symbol table’ rather than ‘dictionary’ when referring to the abstract data type (ADT).

13.2 What are Symbol Tables?

We can define the *symbol table* as a data structure that associates a *value* with a *key*. It supports the following operations:

- Search whether a particular name is in the table
- Get the attributes of that name
- Modify the attributes of that name
- Insert a new name and its attributes
- Delete a name and its attributes

There are only three basic operations on symbol tables: searching, inserting, and deleting.

Example: DNS lookup. Let us assume that the key in this case is the URL and the value is an IP address.

- Insert URL with specified IP address
- Given URL, find corresponding IP address

Key[Website]	Value [IP Address]
www.CareerMonks.com	128.112.136.11
www.AuthorsInn.com	128.112.128.15
www.AuthInn.com	130.132.143.21
www.klm.com	128.103.060.55
www.CareerMonk.com	209.052.165.60

13.3 Symbol Table Implementations

Before implementing symbol tables, let us enumerate the possible implementations. Symbol tables can be implemented in many ways and some of them are listed below.

Unordered Array Implementation

With this method, just maintaining an array is enough. It needs $O(n)$ time for searching, insertion and deletion in the worst case.

Ordered [Sorted] Array Implementation

In this we maintain a sorted array of keys and values.

- Store in sorted order by key
- $\text{keys}[i] = i^{\text{th}}$ largest key
- $\text{values}[i] = \text{value associated with } i^{\text{th}} \text{ largest key}$

Since the elements are sorted and stored in arrays, we can use a simple binary search for finding an element. It takes $O(\log n)$ time for searching and $O(n)$ time for insertion and deletion in the worst case.

Unordered Linked List Implementation

Just maintaining a linked list with two data values is enough for this method. It needs $O(n)$ time for searching, insertion and deletion in the worst case.

Ordered Linked List Implementation

In this method, while inserting the keys, maintain the order of keys in the linked list. Even if the list is sorted, in the worst case it needs $O(n)$ time for searching, insertion and deletion.

Binary Search Trees Implementation

Refer to [Trees](#) chapter. The advantages of this method are: it does not need much code and it has a fast search [$O(\log n)$ on average].

Balanced Binary Search Trees Implementation

Refer to [Trees](#) chapter. It is an extension of binary search trees implementation and takes $O(\log n)$ in worst case for search, insert and delete operations.

Ternary Search Implementation

Refer to [String Algorithms](#) chapter. This is one of the important methods used for implementing dictionaries.

Hashing Implementation

This method is important. For a complete discussion, refer to the [Hashing](#) chapter.

13.4 Comparison Table of Symbols for Implementations

Let us consider the following comparison table for all the implementations.

Implementation	Search	Insert	Delete
Unordered Array	n	n	n
Ordered Array (can be implemented with array binary search)	$\log n$	n	n
Unordered List	n	n	n
Ordered List	n	n	n
Binary Search Trees ($O(\log n)$ on average)	$\log n$	$\log n$	$\log n$
Balanced Binary Search Trees ($O(\log n)$ in worst case)	$\log n$	$\log n$	$\log n$
Ternary Search (only change is in logarithms base)	$\log n$	$\log n$	$\log n$
Hashing ($O(1)$ on average)	1	1	1

Notes:

- In the above table, n is the input size.
- Table indicates the possible implementations discussed in this book. But, there could be other implementations.

HASHING

CHAPTER

14



14.1 What is Hashing?

Hashing is a technique used for storing and retrieving information as quickly as possible. It is used to perform optimal searches and is useful in implementing symbol tables.

14.2 Why Hashing?

In the *Trees* chapter we saw that balanced binary search trees support operations such as *insert*, *delete* and *search* in $O(\log n)$ time. In applications, if we need these operations in $O(1)$, then hashing provides a way. Remember that worst case complexity of hashing is still $O(n)$, but it gives $O(1)$ on the average.

14.3 HashTable ADT

The common operations for hash table are:

- CreatHashTable: Creates a new hash table
- HashSearch: Searches the key in hash table
- HashInsert: Inserts a new key into hash table
- HashDelete: Deletes a key from hash table
- DeleteHashTable: Deletes the hash table

14.4 Understanding Hashing

In simple terms we can treat *array* as a hash table. For understanding the use of hash tables, let us consider the following example: Give an algorithm for printing the first repeated character if there are duplicated elements in it. Let us think about the possible solutions. The simple and brute force way of solving is: given a string, for each character check whether that character is repeated or not. The time complexity of this approach is $O(n^2)$ with $O(1)$ space complexity.

Now, let us find a better solution for this problem. Since our objective is to find the first repeated character, what if we remember the previous characters in some array?

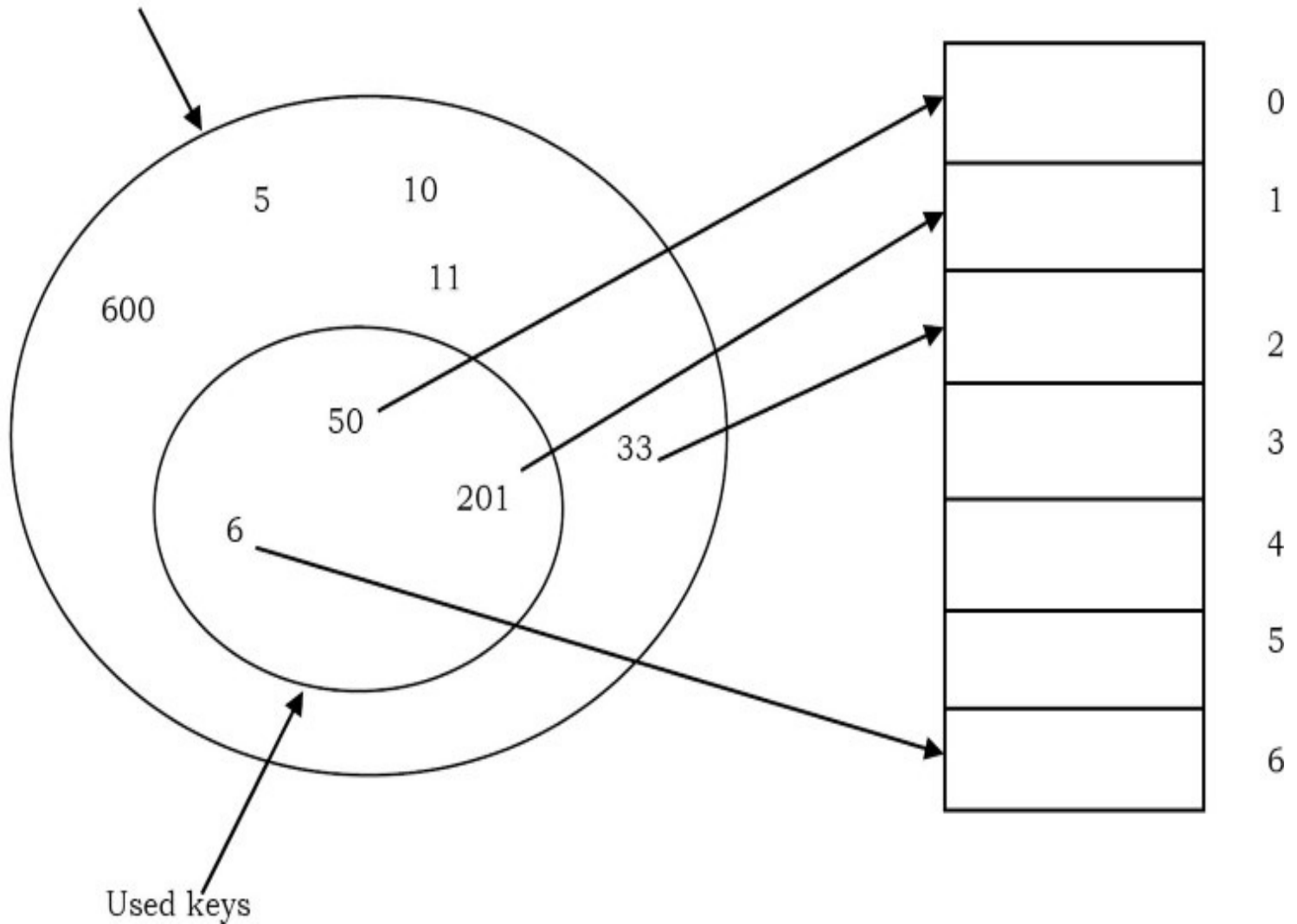
We know that the number of possible characters is 256 (for simplicity assume *ASCII* characters only). Create an array of size 256 and initialize it with all zeros. For each of the input characters go to the corresponding position and increment its count. Since we are using arrays, it takes constant time for reaching any location. While scanning the input, if we get a character whose counter is already 1 then we can say that the character is the one which is repeating for the first time.

```
char FirstRepeatedChar ( char *str ) {
    int i, len=strlen(str);
    int count[256]; //additional array
    for(i=0; i<256; ++i)
        count[i] = 0;
    for(i=0; i<len; ++i) {
        if(count[str[i]]==1) {
            printf("%c", str[i]);
            break;
        }
        else count[str[i]]++;
    }
    if(i==len)
        printf("No Repeated Characters");
    return 0;
}
```

Why not Arrays?

In the previous problem, we have used an array of size 256 because we know the number of different possible characters [256] in advance. Now, let us consider a slight variant of the same problem. Suppose the given array has numbers instead of characters, then how do we solve the problem?

Universe of possible keys



In this case the set of possible values is infinity (or at least very big). Creating a huge array and storing the counters is not possible. That means there are a set of universal keys and limited locations in the memory. If we want to solve this problem we need to somehow map all these possible keys to the possible memory locations. From the above discussion and diagram it can be seen that we need a mapping of possible keys to one of the available locations. As a result using simple arrays is not the correct choice for solving the problems where the possible keys are very big. The process of mapping the keys to locations is called *hashing*.

Note: For now, do not worry about how the keys are mapped to locations. That depends on the function used for conversions. One such simple function is $key \% \text{table size}$.

14.5 Components of Hashing

Hashing has four key components:

- 1) Hash Table
- 2) Hash Functions
- 3) Collisions
- 4) Collision Resolution Techniques

14.6 Hash Table

Hash table is a generalization of array. With an array, we store the element whose key is k at a position k of the array. That means, given a key k , we find the element whose key is k by just looking in the k^{th} position of the array. This is called *direct addressing*.

Direct addressing is applicable when we can afford to allocate an array with one position for every possible key. But if we do not have enough space to allocate a location for each possible key, then we need a mechanism to handle this case. Another way of defining the scenario is: if we have less locations and more possible keys, then simple array implementation is not enough.

In these cases one option is to use hash tables. Hash table or hash map is a data structure that stores the keys and their associated values, and hash table uses a hash function to map keys to their associated values. The general convention is that we use a hash table when the number of keys actually stored is small relative to the number of possible keys.

14.7 Hash Function

The hash function is used to transform the key into the index. Ideally, the hash function should map each possible key to a unique slot index, but it is difficult to achieve in practice.

Given a collection of elements, a hash function that maps each item into a unique slot is referred to as a *perfect hash function*. If we know the elements and the collection will never change, then it is possible to construct a perfect hash function. Unfortunately, given an arbitrary collection of elements, there is no systematic way to construct a perfect hash function. Luckily, we do not need the hash function to be perfect to still gain performance efficiency.

One way to always have a perfect hash function is to increase the size of the hash table so that each possible value in the element range can be accommodated. This guarantees that each element will have a unique slot. Although this is practical for small numbers of elements, it is not feasible when the number of possible elements is large. For example, if the elements were nine-digit Social Security numbers, this method would require almost one billion slots. If we only want to store data for a class of 25 students, we will be wasting an enormous amount of memory.

Our goal is to create a hash function that minimizes the number of collisions, is easy to compute, and evenly distributes the elements in the hash table. There are a number of common ways to extend the simple remainder method. We will consider a few of them here.

The *folding method* for constructing hash functions begins by dividing the elements into equal-size pieces (the last piece may not be of equal size). These pieces are then added together to give the resulting hash value. For example, if our element was the phone number 436-555-4601, we would take the digits and divide them into groups of 2 (43,65,55,46,01). After the addition, $43+65+55+46+01$, we get 210. If we assume our hash table has 11 slots, then we need to perform the extra step of dividing by 11 and keeping the remainder. In this case $210 \% 11$ is 1, so the phone number 436-555-4601 hashes to slot 1. Some folding methods go one step further and reverse every other piece before the addition. For the above example, we get $43+56+55+64+01=219$ which gives $219 \% 11 = 10$.

How to Choose Hash Function?

The basic problems associated with the creation of hash tables are:

- An efficient hash function should be designed so that it distributes the index values of inserted objects uniformly across the table.
- An efficient collision resolution algorithm should be designed so that it computes an alternative index for a key whose hash index corresponds to a location previously inserted in the hash table.
- We must choose a hash function which can be calculated quickly, returns values within the range of locations in our table, and minimizes collisions.

Characteristics of Good Hash Functions

A good hash function should have the following characteristics:

- Minimize collision
- Be easy and quick to compute
- Distribute key values evenly in the hash table
- Use all the information provided in the key
- Have a high load factor for a given set of keys

14.8 Load Factor

The load factor of a non-empty hash table is the number of items stored in the table divided by the size of the table. This is the decision parameter used when we want to rehash or expand the existing hash table entries. This also helps us in determining the efficiency of the hashing function. That means, it tells whether the hash function is distributing the keys uniformly or not.

$$\text{Load factor} = \frac{\text{Number of elements in hash table}}{\text{Hash Table size}}$$

14.9 Collisions

Hash functions are used to map each key to a different address space, but practically it is not possible to create such a hash function and the problem is called *collision*. Collision is the condition where two records are stored in the same location.

14.10 Collision Resolution Techniques

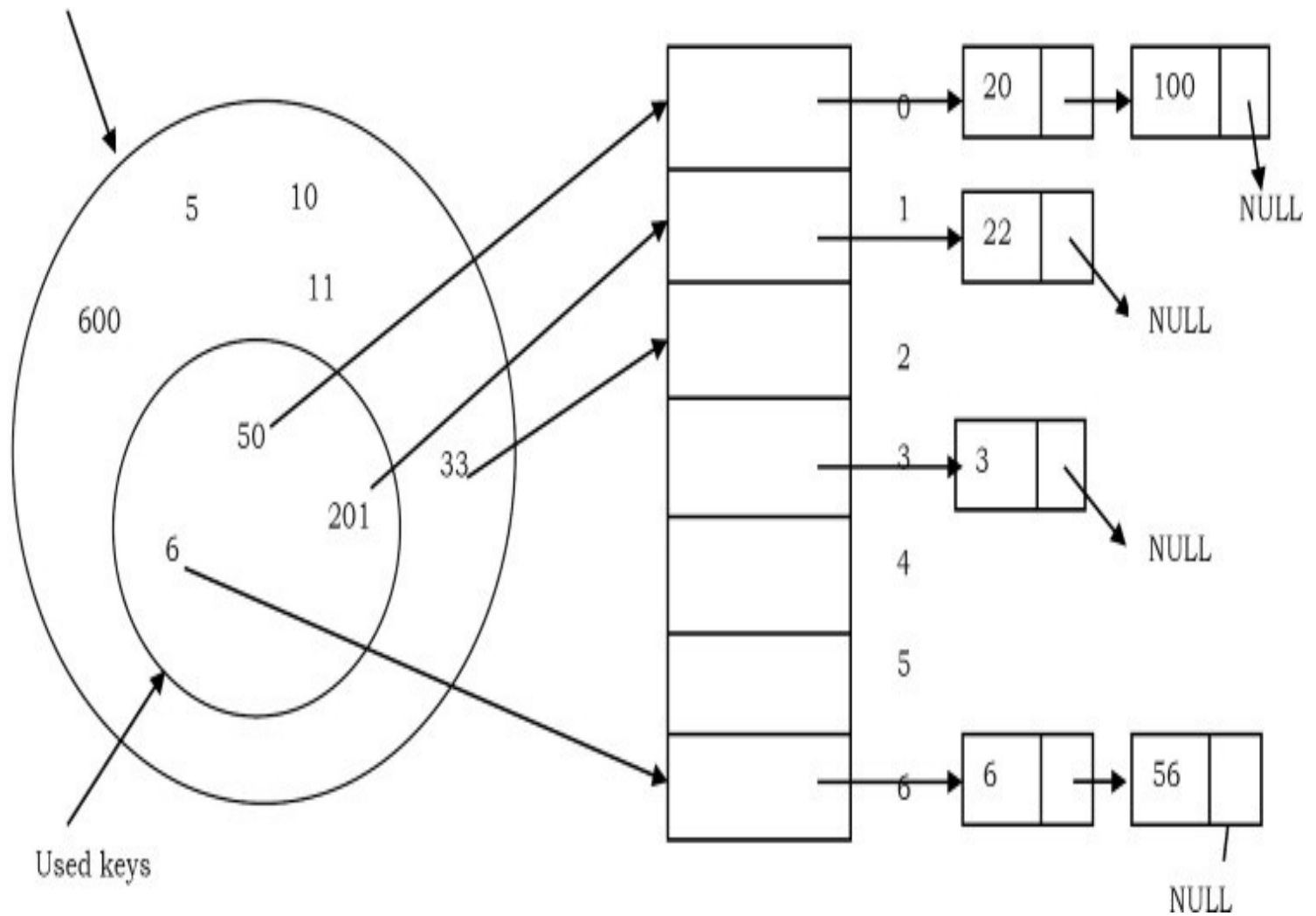
The process of finding an alternate location is called *collision resolution*. Even though hash tables have collision problems, they are more efficient in many cases compared to all other data structures, like search trees. There are a number of collision resolution techniques, and the most popular are direct chaining and open addressing.

- **Direct Chaining:** An array of linked list application
 - Separate chaining
- **Open Addressing:** Array-based implementation
 - Linear probing (linear search)
 - Quadratic probing (*nonlinear search*)
 - Double hashing (use two hash functions)

14.11 Separate Chaining

Collision resolution by chaining combines linked representation with hash table. When two or more records hash to the same location, these records are constituted into a singly-linked list called a *chain*.

Universe of possible keys



14.12 Open Addressing

In open addressing all keys are stored in the hash table itself. This approach is also known as *closed hashing*. This procedure is based on probing. A collision is resolved by probing.

Linear Probing

The interval between probes is fixed at 1. In linear probing, we search the hash table sequentially, starting from the original hash location. If a location is occupied, we check the next location. We wrap around from the last table location to the first table location if necessary. The function for rehashing is the following:

$$rehash(key) = (n + 1) \% tablesize$$

One of the problems with linear probing is that table items tend to cluster together in the hash table. This means that the table contains groups of consecutively occupied locations that are

called *clustering*.

Clusters can get close to one another, and merge into a larger cluster. Thus, the one part of the table might be quite dense, even though another part has relatively few items. Clustering causes long probe searches and therefore decreases the overall efficiency.

The next location to be probed is determined by the step-size, where other step-sizes (more than one) are possible. The step-size should be relatively prime to the table size, i.e. their greatest common divisor should be equal to 1. If we choose the table size to be a prime number, then any step-size is relatively prime to the table size. Clustering cannot be avoided by larger step-sizes.

Quadratic Probing

The interval between probes increases proportionally to the hash value (the interval thus increasing linearly, and the indices are described by a quadratic function). The problem of Clustering can be eliminated if we use the quadratic probing method.

In quadratic probing, we start from the original hash location i . If a location is occupied, we check the locations $i + 1^2$, $i + 2^2$, $i + 3^2$, $i + 4^2$... We wrap around from the last table location to the first table location if necessary. The function for rehashing is the following:

$$rehash(key) = (n + k^2) \% tablesize$$

Example: Let us assume that the table size is 11 (0..10)

Hash Function: $h(key) = key \bmod 11$

0	
1	
2	2
3	13
4	25
5	5
6	24
7	9
8	19
9	31
10	21

Insert keys

$$31 \bmod 11 = 9$$

$$19 \bmod 11 = 8$$

$$2 \bmod 11 = 2$$

$$13 \bmod 11 = 2 \rightarrow 2 + 1^2 = 3$$

$$25 \bmod 11 = 3 \rightarrow 3 + 1^2 = 4$$

$$24 \bmod 11 = 2 \rightarrow 2 + 1^2, 2 + 2^2 = 6$$

$$21 \bmod 11 = 10$$

$$9 \bmod 11 = 9 \rightarrow 9 + 1^2, 9 + 2^2 \bmod 11, 9 + 3^2 \bmod 11 = 7$$

Even though clustering is avoided by quadratic probing, still there are chances of clustering. Clustering is caused by multiple search keys mapped to the same hash key. Thus, the probing sequence for such search keys is prolonged by repeated conflicts along the probing sequence. Both linear and quadratic probing use a probing sequence that is independent of the search key.

Double Hashing

The interval between probes is computed by another hash function. Double hashing reduces clustering in a better way. The increments for the probing sequence are computed by using a second hash function. The second hash function $h2$ should be:

$$h2(key) \neq 0 \text{ and } h2 \neq h1$$

We first probe the location $h1(key)$. If the location is occupied, we probe the location $h1(key) + h2(key)$, $h1(key) + 2 * h2(key)$, ...

Example:

Table size is 11 (0..10)

Hash Function: assume $h1(key) = key \bmod 11$ and $h2(key) = 7 - (key \bmod 7)$

0	
1	
2	
3	58
4	25
5	
6	91
7	
8	
9	25
10	14

Insert keys:

$58 \bmod 11 = 3$

$14 \bmod 11 = 3 \rightarrow 3 + 7 = 10$

$91 \bmod 11 = 3 \rightarrow 3 + 7, 3 + 2 * 7 \bmod 11 = 6$

$25 \bmod 11 = 3 \rightarrow 3 + 3, 3 + 2 * 3 = 9$

14.13 Comparison of Collision Resolution Techniques

Comparisons: Linear Probing vs. Double Hashing

The choice between linear probing and double hashing depends on the cost of computing the hash function and on the load factor [number of elements per slot] of the table. Both use few probes but double hashing take more time because it hashes to compare two hash functions for long keys.

Comparisons: Open Addressing vs. Separate Chaining

It is somewhat complicated because we have to account for the memory usage. Separate chaining uses extra memory for links. Open addressing needs extra memory implicitly within the table to terminate the probe sequence. Open-addressed hash tables cannot be used if the data does not have unique keys. An alternative is to use separate chained hash tables.

Comparisons: Open Addressing methods

Linear Probing	Quadratic Probing	Double hashing
Fastest among three	Easiest to implement and deploy	Makes more efficient use of memory
Uses few probes	Uses extra memory for links and it does not probe all locations in the table	Uses few probes but takes more time
A problem occurs known as primary clustering	A problem occurs known as secondary clustering	More complicated to implement
Interval between probes is fixed - often at 1.	Interval between probes increases proportional to the hash value	Interval between probes is computed by another hash function

14.14 How Hashing Gets $O(1)$ Complexity

From the previous discussion, one doubts how hashing gets $O(1)$ if multiple elements map to the same location...

The answer to this problem is simple. By using the load factor we make sure that each block (for example, linked list in separate chaining approach) on the average stores the maximum number of elements less than the *load factor*. Also, in practice this load factor is a constant (generally, 10 or 20). As a result, searching in 20 elements or 10 elements becomes constant.

If the average number of elements in a block is greater than the load factor, we rehash the elements with a bigger hash table size. One thing we should remember is that we consider average occupancy (total number of elements in the hash table divided by table size) when deciding the rehash.

The access time of the table depends on the load factor which in turn depends on the hash function. This is because hash function distributes the elements to the hash table. For this reason, we say hash table gives $O(1)$ complexity on average. Also, we generally use hash tables in cases

where searches are more than insertion and deletion operations.

14.15 Hashing Techniques

There are two types of hashing techniques: static hashing and dynamic hashing

Static Hashing

If the data is fixed then static hashing is useful. In static hashing, the set of keys is kept fixed and given in advance, and the number of primary pages in the directory are kept fixed.

Dynamic Hashing

If the data is not fixed, static hashing can give bad performance, in which case dynamic hashing is the alternative, in which case the set of keys can change dynamically.

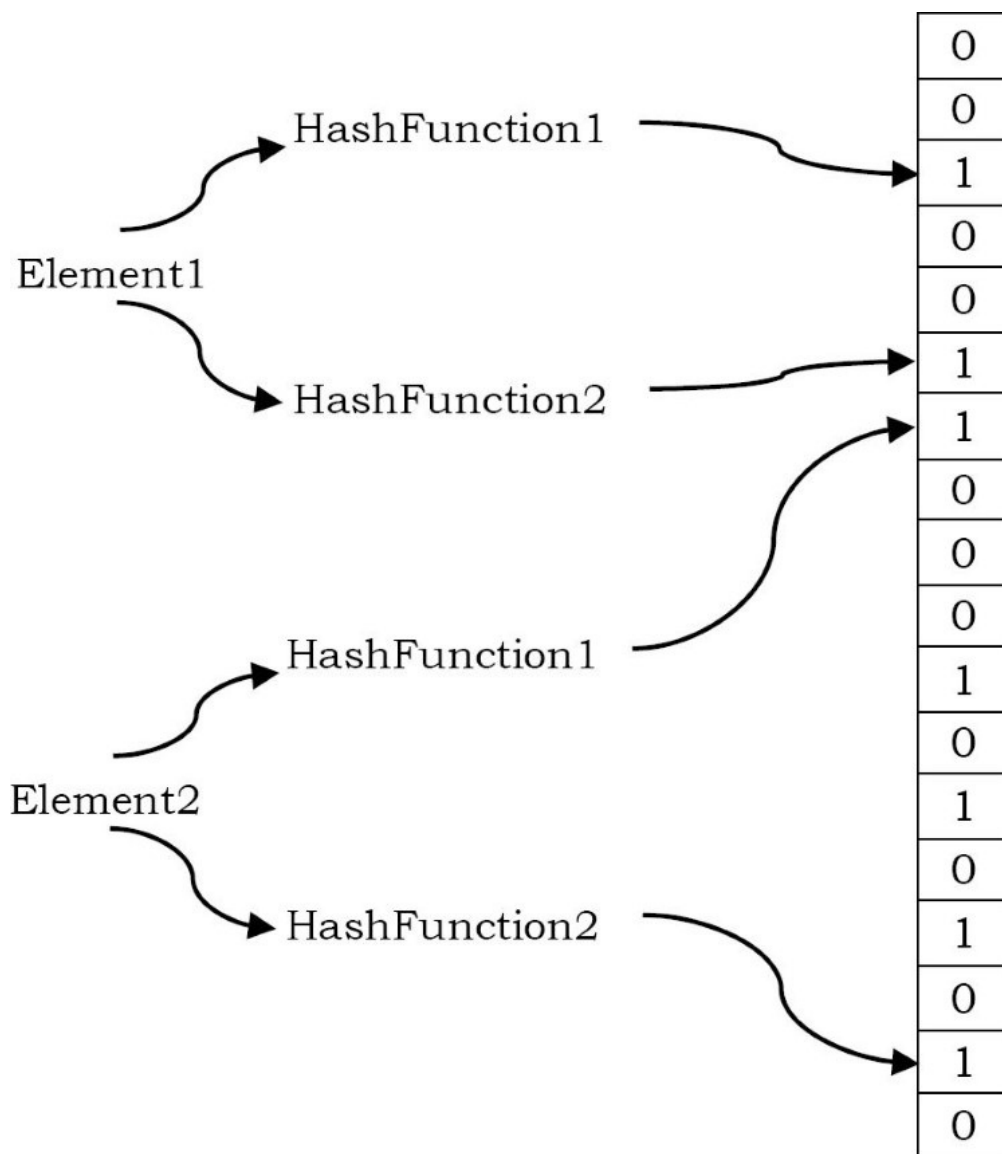
14.16 Problems for which Hash Tables are not suitable

- Problems for which data ordering is required
- Problems having multidimensional data
- Prefix searching, especially if the keys are long and of variable-lengths
- Problems that have dynamic data
- Problems in which the data does not have unique keys.

14.17 Bloom Filters

A Bloom filter is a probabilistic data structure which was designed to check whether an element is present in a set with memory and time efficiency. It tells us that the element either definitely is *not* in the set or *may* be in the set. The base data structure of a Bloom filter is a *Bit Vector*. The algorithm was invented in 1970 by Burton Bloom and it relies on the use of a number of different hash functions.

How it works?



Now that the bits in the bit vector have been set for *Element1* and *Element2*; we can query the bloom filter to tell us if something has been seen before.

The element is hashed but instead of setting the bits, this time a check is done and if the bits that would have been set are already set the bloom filter will return true that the element has been seen before.

A Bloom filter starts off with a bit array initialized to zero. To store a data value, we simply apply k different hash functions and treat the resulting k values as indices in the array, and we set each of the k array elements to 1. We repeat this for every element that we encounter.

Now suppose an element turns up and we want to know if we have seen it before. What we do is apply the k hash functions and look up the indicated array elements. If any of them are 0 we can be 100% sure that we have never encountered the element before - if we had, the bit would have been set to 1. However, even if all of them are one, we still can't conclude that we have seen the element before because all of the bits could have been set by the k hash functions applied to multiple other elements. All we can conclude is that it is *likely* that we have encountered the

element before.

Note that it is not possible to remove an element from a Bloom filter. The reason is simply that we can't unset a bit that appears to belong to an element because it might also be set by another element.

If the bit array is mostly empty, i.e., set to zero, and the k hash functions are independent of one another, then the probability of a false positive (i.e., concluding that we have seen a data item when we actually haven't) is low. For example, if there are only k bits set, we can conclude that the probability of a false positive is very close to zero as the only possibility of error is that we entered a data item that produced the same k hash values - which is unlikely as long as the 'has' functions are independent.

As the bit array fills up, the probability of a false positive slowly increases. Of course when the bit array is full, every element queried is identified as having been seen before. So clearly we can trade space for accuracy as well as for time.

One-time removal of an element from a Bloom filter can be simulated by having a second Bloom filter that contains elements that have been removed. However, false positives in the second filter become false negatives in the composite filter, which may be undesirable. In this approach, re-adding a previously removed item is not possible, as one would have to remove it from the *removed* filter.

Selecting hash functions

The requirement of designing k different independent hash functions can be prohibitive for large k . For a good hash function with a wide output, there should be little if any correlation between different bit-fields of such a hash, so this type of hash can be used to generate multiple *different* hash functions by slicing its output into multiple bit fields. Alternatively, one can pass k different initial values (such as 0, 1, ..., $k - 1$) to a hash function that takes an initial value – or add (or append) these values to the key. For larger m and/or k , independence among the hash functions can be relaxed with negligible increase in the false positive rate.

Selecting size of bit vector

A Bloom filter with 1% error and an optimal value of k , in contrast, requires only about 9.6 bits per element – regardless of the size of the elements. This advantage comes partly from its compactness, inherited from arrays, and partly from its probabilistic nature. The 1% false-positive rate can be reduced by a factor of ten by adding only about 4.8 bits per element.

Space Advantages

While risking false positives, Bloom filters have a strong space advantage over other data structures for representing sets, such as self-balancing binary search trees, tries, hash tables, or simple arrays or linked lists of the entries. Most of these require storing at least the data items themselves, which can require anywhere from a small number of bits, for small integers, to an arbitrary number of bits, such as for strings (tries are an exception, since they can share storage between elements with equal prefixes). Linked structures incur an additional linear space overhead for pointers.

However, if the number of potential values is small and many of them can be in the set, the Bloom filter is easily surpassed by the deterministic bit array, which requires only one bit for each potential element.

Time Advantages

Bloom filters also have the unusual property that the time needed either to add items or to check whether an item is in the set is a fixed constant, $O(k)$, completely independent of the number of items already in the set. No other constant-space set data structure has this property, but the average access time of sparse hash tables can make them faster in practice than some Bloom filters. In a hardware implementation, however, the Bloom filter shines because its k lookups are independent and can be parallelized.

Implementation

Refer to *Problems Section*.

14.18 Hashing: Problems & Solutions

Problem-1 Implement a separate chaining collision resolution technique. Also, discuss time complexities of each function.

Solution: To create a hashtable of given size, say n , we allocate an array of n/L (whose value is usually between 5 and 20) pointers to list, initialized to NULL. To perform *Search/Insert/Delete* operations, we first compute the index of the table from the given key by using *hashfunction* and then do the corresponding operation in the linear list maintained at that location. To get uniform distribution of keys over a hashtable, maintain table size as the prime number.


```

#define LOAD_FACTOR 20
struct ListNode {
    int key;
    int data;
    struct ListNode *next;
};
struct HashTableNode {
    int bcount;           //Number of elements in block
    struct ListNode *next;
};
struct HashTable {
    int tsize;
    int count;           //Number of elements in table
    struct HashTableNode **Table;
};
struct HashTable *CreatHashTable(int size) {
    struct HashTable *h;
    h = (struct HashTable *)malloc(sizeof(struct HashTable));
    if(!h)
        return NULL;
    h->tsize = size/ LOAD_FACTOR;
    h->count = 0;
    h->Table = (struct HashTableNode **) malloc( sizeof(struct HashTableNode *) * h->tsize);
    if(!h->Table) {
        printf("Memory Error");
        return NULL;
    }
    for(int i=0; i < h->tsize; i++) {
        h->Table[i]->next = NULL;
        h->Table[i]->bcount = 0;
    }
    return h;
}
int HashSearch(Struct HashTable *h, int data) {
    struct ListNode *temp;
    temp = h->Table[Hash(data, h->tsize)]->next;           // Assume Hash is a built-in function
    while(temp) {
        if(temp->data == data)
            return 1;
        temp = temp->next;
    }
    return 0;
}
int HashInsert(Struct HashTable *h, int data) {
    int index;
    struct ListNode *temp, *newNode;
    if(HashSearch(h, data))
        return 0;
    index = Hash(data, h->tsize);           // Assume Hash is a built-in function
    temp = h->Table[index]->next;
    newNode = (struct ListNode *) malloc(sizeof(struct ListNode));
    if(!newNode) {
        printf("Out of Space");
        return -1;
    }
    newNode->key = index;
    newNode->data = data;
    newNode->next = h->Table[index]->next;
}

```



```

    h→Table[index]→next = newNode;
    h→Table[index]→bcount++;
    h→count++;
    if(h→count / h→tsize > LOAD_FACTOR)
        Rehash(h);
    return 1;
}

int HashDelete(Struct HashTable *h, int data) {
    int index;
    struct ListNode *temp, *prev;
    index = Hash(data, h→tsize);
    for(temp = h→Table[index]→next, prev = NULL; temp; prev = temp, temp = temp→next) {
        if(temp→data == data) {
            if(prev != NULL)
                prev→next = temp→next;
            free(temp);
            h→Table[index]→bcount--;
            h→count--;
            return 1;
        }
    }
    return 0;
}

void Rehash(Struct HashTable *h) {
    int oldsize, i, index;
    struct ListNode *p, *temp, *temp2;
    struct HashTableNode **oldTable;
    oldsize = h→tsize;
    oldTable = h→Table;
    h→tsize = h→tsize * 2;
    h→Table = (struct HashTableNode **) malloc(h→tsize * sizeof(struct HashTableNode *));
    if(!h→Table) {
        printf("Allocation Failed");
        return;
    }
    for(i = 0; i < oldsize; i++) {
        for(temp = oldTable[i]→next; temp; temp = temp→next) {
            index = Hash(temp→data, h→tsize);
            temp2 = temp; temp = temp→next;
            temp2→next = h→Table[index]→next;
            h→Table[index]→next = temp2;
        }
    }
}

```

CreatHashTable – $O(n)$. HashSearch - $O(1)$ average. HashInsert - $O(1)$ average. HashDelete - $O(1)$ average.

Problem-2 Given an array of characters, give an algorithm for removing the duplicates.

Solution: Start with the first character and check whether it appears in the remaining part of the string using a simple linear search. If it repeats, bring the last character to that position and decrement the size of the string by one. Continue this process for each distinct character of the given string.

```
int elem(int *A, size_t n, int e){
    for (int i = 0; i < n; ++i)
        if (A[i] == e)
            return 1;
    return 0;
}

int RemoveDuplicates(int *A, int n){
    int m = 0;
    for (int i = 0; i < n; ++i)
        if (!elem(A, m, A[i]))
            A[m++] = A[i];
    return m;
}
```

Time Complexity: $O(n^2)$. Space Complexity: $O(1)$.

Problem-3 Can we find any other idea to solve this problem in better time than $O(n^2)$?
Observe that the order of characters in solutions do not matter.

Solution: Use sorting to bring the repeated characters together. Finally scan through the array to remove duplicates in consecutive positions.

```

int Compare(const void* a, const void *b) {
    return *(char*)a - *(char*)b;
}

void RemoveDuplicates(char s[]) {
    int last, current;
    QuickSort(s, strlen(s), sizeof(char), Compare);
    current = 0, last = 0;
    for(; s[current]; i++) {
        if(s[last] != s[current])
            s[++last] = s[current];
    }
    s[last] = '\0';
}

```

Time Complexity: $\Theta(n \log n)$. Space Complexity: $O(1)$.

Problem-4 Can we solve this problem in a single pass over given array?

Solution: We can use hash table to check whether a character is repeating in the given string or not. If the current character is not available in hash table, then insert it into hash table and keep that character in the given string also. If the current character exists in the hash table then skip that character.

```

void RemoveDuplicates(char s[]) {
    int src, dst;
    struct Hashtable *h;
    h = CreateHashTable();
    current = last = 0;
    for(; s[current]; current++) {
        if( !HashSearch(h, s[current])) {
            s[last++] = s[current];
            HashInsert(h, s[current]);
        }
    }
    s[last] = '\0';
}

```

Time Complexity: $\Theta(n)$ on average. Space Complexity: $O(n)$.

Problem-5 Given two arrays of unordered numbers, check whether both arrays have the same set of numbers?

Solution: Let us assume that two given arrays are A and B. A simple solution to the given

problem is: for each element of A, check whether that element is in B or not. A problem arises with this approach if there are duplicates. For example consider the following inputs:

$$A = \{2, 5, 6, 8, 10, 2, 2\}$$
$$B = \{2, 5, 5, 8, 10, 5, 6\}$$

The above algorithm gives the wrong result because for each element of A there is an element in B also. But if we look at the number of occurrences, they are not the same. This problem we can solve by moving the elements which are already compared to the end of the list. That means, if we find an element in B, then we move that element to the end of B, and in the next searching we will not find those elements. But the disadvantage of this is it needs extra swaps. Time Complexity of this approach is $O(n^2)$, since for each element of A we have to scan B.

Problem-6 Can we improve the time complexity of [Problem-5](#)?

Solution: Yes. To improve the time complexity, let us assume that we have sorted both the lists. Since the sizes of both arrays are n, we need $O(n \log n)$ time for sorting them. After sorting, we just need to scan both the arrays with two pointers and see whether they point to the same element every time, and keep moving the pointers until we reach the end of the arrays.

Time Complexity of this approach is $O(n \log n)$. This is because we need $O(n \log n)$ for sorting the arrays. After sorting, we need $O(n)$ time for scanning but it is less compared to $O(n \log n)$.

Problem-7 Can we further improve the time complexity of [Problem-5](#)?

Solution: Yes, by using a hash table. For this, consider the following algorithm.

Algorithm:

- Construct the hash table with array A elements as keys.
- While inserting the elements, keep track of the number frequency for each number. That means, if there are duplicates, then increment the counter of that corresponding key.
- After constructing the hash table for A's elements, now scan the array B.
- For each occurrence of B's elements reduce the corresponding counter values.
- At the end, check whether all counters are zero or not.
- If all counters are zero, then both arrays are the same otherwise the arrays are different.

Time Complexity; $O(n)$ for scanning the arrays. Space Complexity; $O(n)$ for hash table.

Problem-8 Given a list of number pairs; if $pair(i, j)$ exists, and $pair(j, i)$ exists, report all such pairs. For example, in $\{\{1, 3\}, \{2, 6\}, \{3, 5\}, \{7, 4\}, \{5, 3\}, \{8, 7\}\}$, we see that $\{3, 5\}$ and $\{5, 3\}$ are present. Report this pair when you encounter $\{5, 3\}$. We call such pairs 'symmetric pairs'. So, give an efficient algorithm for finding all such pairs.

Solution: By using hashing, we can solve this problem in just one scan. Consider the following algorithm.

Algorithm:

- Read the pairs of elements one by one and insert them into the hash table. For each pair, consider the first element as key and the second element as value.
- While inserting the elements, check if the hashing of the second element of the current pair is the same as the first number of the current pair.
- If they are the same, then that indicates a symmetric pair exists and output that pair.
- Otherwise, insert that element into that. That means, use the first number of the current pair as key and the second number as value and insert them into the hash table.
- By the time we complete the scanning of all pairs, we have output all the symmetric pairs.

Time Complexity; $O(n)$ for scanning the arrays. Note that we are doing a scan only of the input.
Space Complexity; $O(n)$ for hash table.

Problem-9 Given a singly linked list, check whether it has a loop in it or not.

Solution: Using Hash Tables

Algorithm:

- Traverse the linked list nodes one by one.
- Check if the node's address is there in the hash table or not.
- If it is already there in the hash table, that indicates we are visiting a node which was already visited. This is possible only if the given linked list has a loop in it.
- If the address of the node is not there in the hash table, then insert that node's address into the hash table.
- Continue this process until we reach the end of the linked list or we find the loop.

Time Complexity; $O(n)$ for scanning the linked list. Note that we are doing a scan only of the input. Space Complexity; $O(n)$ for hash table.

Note: for an efficient solution, refer to the [Linked Lists](#) chapter.

Problem-10 Given an array of 101 elements. Out of them 50 elements are distinct, 24 elements are repeated 2 times, and one element is repeated 3 times. Find the element that is repeated 3 times in $O(1)$.

Solution: Using Hash Tables

Algorithm:

- Scan the input array one by one.
- Check if the element is already there in the hash table or not.
- If it is already there in the hash table, increment its counter value [this indicates the number of occurrences of the element].
- If the element is not there in the hash table, insert that node into the hash table with counter value 1.
- Continue this process until reaching the end of the array.

Time Complexity: $O(n)$, because we are doing two scans. Space Complexity: $O(n)$, for hash table.

Note: For an efficient solution refer to the [Searching](#) chapter.

Problem-11 Given m sets of integers that have n elements in them, provide an algorithm to find an element which appeared in the maximum number of sets?

Solution: Using Hash Tables

Algorithm:

- Scan the input sets one by one.
- For each element keep track of the counter. The counter indicates the frequency of occurrences in all the sets.
- After completing the scan of all the sets, select the one which has the maximum counter value.

Time Complexity: $O(mn)$, because we need to scan all the sets. Space Complexity: $O(mn)$, for hash table. Because, in the worst case all the elements may be different.

Problem-12 Given two sets A and B , and a number K , Give an algorithm for finding whether there exists a pair of elements, one from A and one from B , that add up to K .

Solution: For simplicity, let us assume that the size of A is m and the size of B is n .

Algorithm:

- Select the set which has minimum elements.
- For the selected set create a hash table. We can use both key and value as the same.
- Now scan the second array and check whether (K -selected element) exists in the hash table or not.
- If it exists then return the pair of elements.
- Otherwise continue until we reach the end of the set.

Time Complexity: $O(\text{Max}(m,n))$, because we are doing two scans. Space Complexity: $O(\text{Min}(m,n))$, for hash table. We can select the small set for creating the hash table.

Problem-13 Give an algorithm to remove the specified characters from a given string which are given in another string?

Solution: For simplicity, let us assume that the maximum number of different characters is 256. First we create an auxiliary array initialized to 0. Scan the characters to be removed, and for each of those characters we set the value to 1, which indicates that we need to remove that character.

After initialization, scan the input string, and for each of the characters, we check whether that character needs to be deleted or not. If the flag is set then we simply skip to the next character, otherwise we keep the character in the input string. Continue this process until we reach the end of the input string. All these operations we can do in-place as given below.

```
void RemoveChars(char str[], char removeTheseChars[]) {
    int srcInd, destInd;
    int auxi[256]; //additional array
    for(srcInd=0; srcInd<256; srcInd++)
        auxi[srcInd]=0;
    //set true for all characters to be removed
    srcInd=0;
    while(removeTheseChars[srcInd]) {
        auxi[removeTheseChars[srcInd]]=1;
        srcInd++;
    }
    //copy chars unless it must be removed
    srcInd=destInd=0;
    while(str[srcInd++]) {
        if(!auxi[str[srcInd]])
            str[destInd++]=str[srcInd];
    }
}
```

Time Complexity: Time for scanning the characters to be removed + Time for scanning the input array = $O(n) + O(m) \approx O(n)$. Where m is the length of the characters to be removed and n is the length of the input string.

Space Complexity: $O(m)$, length of the characters to be removed. But since we are assuming the maximum number of different characters is 256, we can treat this as a constant. But we should keep in mind that when we are dealing with multi-byte characters, the total number of different characters is much more than 256.

Problem-14 Give an algorithm for finding the first non-repeated character in a string. For example, the first non-repeated character in the string “abzddab” is ‘z’.

Solution: The solution to this problem is trivial. For each character in the given string, we can scan the remaining string if that character appears in it. If it does not appear then we are done with the solution and we return that character. If the character appears in the remaining string, then go to the next character.

```
char FirstNonRepeatedChar( char *str ) {  
    int i, j, repeated = 0;  
    int len = strlen(str);  
    for(i = 0; i < len; i++) {  
        repeated = 0;  
        for(j = 0; j < len; j++) {  
            if( i != j && str[i] == str[j] ) {  
                repeated = 1;  
                break;  
            }  
        }  
        if( repeated == 0 ) // Found the first non-repeated character  
            return str[i];  
    }  
    return "";  
}
```

Time Complexity: $O(n^2)$, for two for loops. Space Complexity: $O(1)$.

Problem-15 Can we improve the time complexity of [Problem-13](#)?

Solution: Yes. By using hash tables we can reduce the time complexity. Create a hash table by reading all the characters in the input string and keeping count of the number of times each character appears. After creating the hash table, we can read the hash table entries to see which element has a count equal to 1. This approach takes $O(n)$ space but reduces the time complexity also to $O(n)$.

```

char FirstNonRepeatedCharUsinghash( char * str ) {
    int i, len=strlen(str);
    int count[256]; //additional array
    for(i=0;i<len;++i)
        count[i] = 0;
    for(i=0;i<len;++i)
        count[str[i]]++;
    for(i=0; i<len; ++i) {
        if(count[str[i]]==1) {
            printf("%c",str[i]);
            break;
        }
    }
    if(i==len)
        printf("No Non-repeated Characters");
    return 0;
}

```

Time Complexity; We have $O(n)$ to create the hash table and another $O(n)$ to read the entries of hash table. So the total time is $O(n) + O(n) = O(2n) \approx O(n)$. Space Complexity: $O(n)$ for keeping the count values.

Problem-16 Given a string, give an algorithm for finding the first repeating letter in a string?

Solution: The solution to this problem is somewhat similar to [Problem-13](#) and [Problem-15](#). The only difference is, instead of scanning the hash table twice we can give the answer in just one scan. This is because while inserting into the hash table we can see whether that element already exists or not. If it already exists then we just need to return that character.

```

char FirstRepeatedCharUsinghash( char * str ) {
    int i, len=strlen(str);
    int count[256]; //additional array
    for(i=0;i<len;++i)
        count[i] = 0;
    for(i=0; i<len; ++i) {
        if(count[str[i]]==1) {
            printf("%s",str[i]);
            break;
        }
        else count[str[i]]++;
    }
    if(i==len)
        printf("No Repeated Characters");
    return 0;
}

```

Time Complexity: We have $O(n)$ for scanning and creating the hash table. Note that we need only one scan for this problem. So the total time is $O(n)$. Space Complexity: $O(n)$ for keeping the count values.

Problem-17 Given an array of n numbers, create an algorithm which displays all pairs whose sum is S .

Solution: This problem is similar to [Problem-12](#). But instead of using two sets we use only one set.

Algorithm:

- Scan the elements of the input array one by one and create a hash table. Both key and value can be the same.
- After creating the hash table, again scan the input array and check whether ($S - \text{selected element}$) exists in the hash table *or not*.
- If it exists then return the pair of elements.
- Otherwise continue and read all the elements of the array.

Time Complexity; We have $O(n)$ to create the hash table and another $O(n)$ to read the entries of the hash table. So the total time is $O(n) + O(n) = O(2n) \approx O(n)$. Space Complexity: $O(n)$ for keeping the count values.

Problem-18 Is there any other way of solving [Problem-17](#)?

Solution: Yes. The alternative solution to this problem involves sorting. First sort the input array. After sorting, use two pointers, one at the starting and another at the ending. Each time add the

values of both the indexes and see if their sum is equal to S . If they are equal then print that pair. Otherwise increase the left pointer if the sum is less than S and decrease the right pointer if the sum is greater than S .

Time Complexity: Time for sorting + Time for scanning = $O(n \log n) + O(n) \approx O(n \log n)$.

Space Complexity: $O(1)$.

Problem-19 We have a file with millions of lines of data. Only two lines are identical; the rest are unique. Each line is so long that it may not even fit in the memory. What is the most efficient solution for finding the identical lines?

Solution: Since a complete line may not fit into the main memory, read the line partially and compute the hash from that partial line. Then read the next part of the line and compute the hash. This time use the previous hash also while computing the new hash value. Continue this process until we find the hash for the complete line. Do this for each line and store all the hash values in a file [or maintain a hash table of these hashes]. If at any point you get same hash value, read the corresponding lines part by part and compare.

Note: Refer to [Searching](#) chapter for related problems.

Problem-20 If h is the hashing function and is used to hash n keys into a table of size s , where $n \leq s$, the expected number of collisions involving a particular key X is :

- (A) less than 1.
- (B) less than n .
- (C) less than s .
- (D) less than $\frac{n}{2}$.

Solution: A.

Problem-21 Implement Bloom Filters

Solution: A Bloom Filter is a data structure designed to tell, rapidly and memory-efficiently, whether an element is present in a set. It is based on a probabilistic mechanism where false positive retrieval results are possible, but false negatives are not. At the end we will see how to tune the parameters in order to minimize the number of false positive results.

Let's begin with a little bit of theory. The idea behind the Bloom filter is to allocate a bit vector of length m , initially all set to 0, and then choose k independent hash functions, $h_1, h_2, \dots, h_k$, each with range $[1..m]$. When an element a is added to the set then the bits at positions $h_1(a), h_2(a), \dots, h_k(a)$ in the bit vector are set to 1. Given a query element q we can test whether it is in the set using the bits at positions $h_1(q), h_2(q), \dots, h_k(q)$ in the vector. If any of these bits is 0 we report that q is not in the set otherwise we report that q is. The thing we have to care about is that in the first case there remains some probability that q is not in the set which could lead us to a false positive response.

```

typedef unsigned int (*hashFunctionPointer)(const char *);

struct Bloom{
    int bloomArraySize;
    unsigned char *bloomArray;
    int nHashFunctions;
    hashFunctionPointer *funcsArray;
};

#define SETBLOOMBIT(a, n) (a[n/CHAR_BIT] |= (1<<(n%CHAR_BIT)))
#define GETBLOOMBIT(a, n) (a[n/CHAR_BIT] & (1<<(n%CHAR_BIT)))

struct Bloom *createBloom(int size, int nHashFunctions, ...){
    struct Bloom *blm;
    va_list l;
    int n;
    if(!(blm=malloc(sizeof(struct Bloom))))
        return NULL;
    if(!(blm->bloomArray=calloc((size+CHAR_BIT-1)/CHAR_BIT, sizeof(char)))) {
        free(blm);
        return NULL;
    }
    if(!(blm->funcsArray=(hashFunctionPointer*)malloc(nHashFunctions*sizeof(hashFunctionPointer)))) {
        free(blm->bloomArray);
        free(blm);
        return NULL;
    }
    va_start(l, nHashFunctions);
    for(n=0; n<nHashFunctions; ++n) {
        blm->funcsArray[n]=va_arg(l, hashFunctionPointer);
    }
    va_end(l);
    blm->nHashFunctions=nHashFunctions;
    blm->bloomArraySize=size;
    return blm;
}

int deleteBloom(struct Bloom *blm){
    free(blm->bloomArray);
    free(blm->funcsArray);
    free(blm);
    return 0;
}

int addElementBloom(struct Bloom *blm, const char *s){
    for(int n=0; n<blm->nHashFunctions; ++n) {
        SETBLOOMBIT(blm->bloomArray, blm->funcsArray[n](s)%blm->bloomArraySize);
    }
    return 0;
}

int checkElementBloom(struct Bloom *blm, const char *s){
    for(int n=0; n<blm->nHashFunctions; ++n) {
        if(!(GETBLOOMBIT(blm->bloomArray, blm->funcsArray[n](s)%blm->bloomArraySize))) return 0;
    }
    return 1;
}

unsigned int shiftAddXORHash(const char *key){
    unsigned int h=0;
    while(*key) h^=(h<<5)+(h>>2)+(unsigned char)*key++;
    return h;
}

unsigned int XORHash(const char *key){
    unsigned int h=0;
    hash_t h=0;
    while(*key) h^=*key++;
    return h;
}

```

```

}

int test(){
    FILE *fp;
    char line[1024];
    char *p;
    struct Bloom *blm;
    if(!(blm=createBloom(1500000, 2, shiftAddXORHash, XORHash))) {
        fprintf(stderr, "ERROR: Could not create Bloom filter\n");
        return -1;
    }
    if(!(fp=fopen("path", "r"))){
        fprintf(stderr, "ERROR: Could not open file %s\n", argv[1]);
        return -1;
    }
    while(fgets(line, 1024, fp)) {
        if((p=strchr(line, '\r')) *p='\0';
        if((p=strchr(line, '\n')) *p='\0';
        addElementBloom(blm, line);
    }
    fclose(fp);
    while(fgets(line, 1024, stdin)) {
        if((p=strchr(line, '\r')) *p='\0';
        if((p=strchr(line, '\n')) *p='\0';
        p=strtok(line, " \t,.;\r\n?!-/()");
        while(p) {
            if(!checkBloom(blm, p)) {
                printf("No match for word \"%s\" \n", p);
            }
            p=strtok(NULL, " \t,.;\r\n?!-/()");
        }
    }
    deleteBloom(blm);
    return 1;
}

```

STRING ALGORITHMS

CHAPTER

15



15.1 Introduction

To understand the importance of string algorithms let us consider the case of entering the URL (Uniform Resource Locator) in any browser (say, Internet Explorer, Firefox, or Google Chrome). You will observe that after typing the prefix of the URL, a list of all possible URLs is displayed. That means, the browsers are doing some internal processing and giving us the list of matching URLs. This technique is sometimes called *auto – completion*.

Similarly, consider the case of entering the directory name in the command line interface (in both *Windows* and *UNIX*). After typing the prefix of the directory name, if we press the *tab* button, we get a list of all matched directory names available. This is another example of auto completion.

In order to support these kinds of operations, we need a data structure which stores the string data efficiently. In this chapter, we will look at the data structures that are useful for implementing string algorithms.

We start our discussion with the basic problem of strings: given a string, how do we search a

substring (pattern)? This is called a *string matching* problem. After discussing various string matching algorithms, we will look at different data structures for storing strings.

15.2 String Matching Algorithms

In this section, we concentrate on checking whether a pattern P is a substring of another string T (T stands for text) or not. Since we are trying to check a fixed string P , sometimes these algorithms are called *exact string matching* algorithms. To simplify our discussion, let us assume that the length of given text T is n and the length of the pattern P which we are trying to match has the length m . That means, T has the characters from 0 to $n - 1$ ($T[0 \dots n - 1]$) and P has the characters from 0 to $m - 1$ ($P[0 \dots m - 1]$). This algorithm is implemented in $C++$ as `strstr()`.

In the subsequent sections, we start with the brute force method and gradually move towards better algorithms.

- Brute Force Method
- Rabin-Karp String Matching Algorithm
- String Matching with Finite Automata
- KMP Algorithm
- Boyer-Moore Algorithm
- Suffix Trees

15.3 Brute Force Method

In this method, for each possible position in the text T we check whether the pattern P matches or not. Since the length of T is n , we have $n - m + 1$ possible choices for comparisons. This is because we do not need to check the last $m - 1$ locations of T as the pattern length is m . The following algorithm searches for the first occurrence of a pattern string P in a text string T .

Algorithm

```
int BruteForceStringMatch (int T[], int n, int P[], int m) {
    for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m && P[j] == T[i + j])
            j = j + 1;
        if (j == m)
            return i;
    }
    return -1;
}
```

Time Complexity: $O((n - m + 1) \times m) \approx O(n \times m)$. Space Complexity: $O(1)$.

15.4 Rabin-Karp String Matching Algorithm

In this method, we will use the hashing technique and instead of checking for each possible position in T , we check only if the hashing of P and the hashing of m characters of T give the same result.

Initially, apply the hash function to the first m characters of T and check whether this result and P 's hashing result is the same or not. If they are not the same, then go to the next character of T and again apply the hash function to m characters (by starting at the second character). If they are the same then we compare those m characters of T with P .

Selecting Hash Function

At each step, since we are finding the hash of m characters of T , we need an efficient hash function. If the hash function takes $O(m)$ complexity in every step, then the total complexity is $O(n \times m)$. This is worse than the brute force method because first we are applying the hash function and also comparing.

Our objective is to select a hash function which takes $O(1)$ complexity for finding the hash of m characters of T every time. Only then can we reduce the total complexity of the algorithm. If the hash function is not good (worst case), the complexity of the Rabin-Karp algorithm is $O((n - m + 1) \times m) \approx O(n \times m)$. If we select a good hash function, the complexity of the Rabin-Karp algorithm complexity is $O(m + n)$. Now let us see how to select a hash function which can compute the hash of m characters of T at each step in $O(1)$.

For simplicity, let's assume that the characters used in string T are only integers. That means, all characters in $T \in \{0,1,2,\dots,9\}$. Since all of them are integers, we can view a string of m consecutive characters as decimal numbers. For example, string '61815' corresponds to the number 61815. With the above assumption, the pattern P is also a decimal value, and let us assume that the decimal value of P is p . For the given text $T[0..n - 1]$, let $t(i)$ denote the decimal value of length- m substring $T[i..i + m - 1]$ for $i = 0, 1, \dots, n - m - 1$. So, $t(i) == p$ if and only if $T[i..i + m - 1] == P[0..m - 1]$.

We can compute p in $O(m)$ time using Horner's Rule as:

$$p = P[m - 1] + 10(P[m - 2] + 10(P[m - 3] + \dots + 10(P[1] + 10P[0])\dots))$$

The code for the above assumption is:

```

value = 0;
for (int i = 0; i < m-1; i++) {
    value = value * 10;
    value = value + P[i];
}

```

We can compute all $t(i)$, for $i = 0, 1, \dots, n - m - 1$ values in a total of $O(n)$ time. The value of $t(0)$ can be similarly computed from $T[0..m-1]$ in $O(m)$ time. To compute the remaining values $t(0)$, $t(1), \dots, t(n - m - 1)$, understand that $t(i + 1)$ can be computed from $t(i)$ in constant time.

$$t(i + 1) = 10 * (t(i) - 10^{m-1} * T[i]) + T[i + m - 1]$$

For example, if $T = "123456"$ and $m = 3$

$$\begin{aligned}
 t(0) &= 123 \\
 t(1) &= 10 * (123 - 100 * 1) + 4 = 234
 \end{aligned}$$

Step by Step explanation

First : remove the first digit : $123 - 100 * 1 = 23$

Second: Multiply by 10 to shift it : $23 * 10 = 230$

Third: Add last digit : $230 + 4 = 234$

The algorithm runs by comparing, $t(i)$ with p . When $t(i) == p$, then we have found the substring P in T , starting from position i .

15.5 String Matching with Finite Automata

In this method we use the finite automata which is the concept of the Theory of Computation (ToC). Before looking at the algorithm, first let us look at the definition of finite automata.

Finite Automata

A finite automaton F is a 5-tuple $(Q, q_0, A, \Sigma, \delta)$, where

- Q is a finite set of states
- $q_0 \in Q$ is the start state
- $A \subseteq Q$ is a set of accepting states
- Σ is a finite input alphabet
- δ is the transition function that gives the next state for a given current state and input

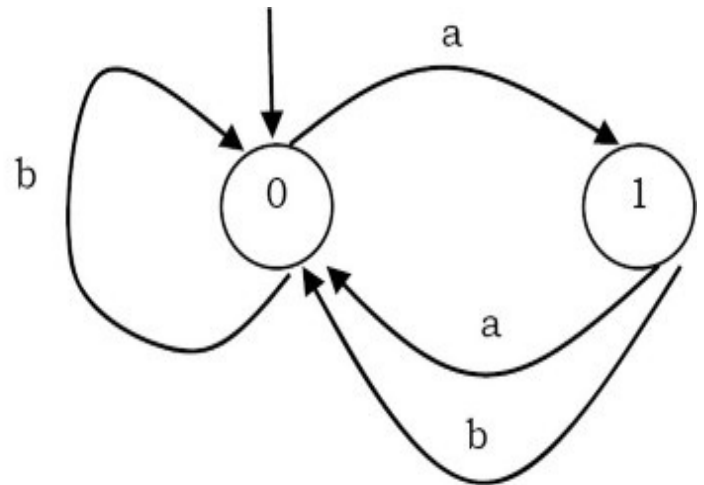
How does Finite Automata Work?

- The finite automaton F begins in state q_0
- Reads characters from Σ one at a time
- If F is in state q and reads input character a , F moves to state $\delta(q,a)$
- At the end, if its state is in A , then we say, F accepted the input string read so far
- If the input string is not accepted it is called the rejected string

Example: Let us assume that $Q = \{0,1\}, q_0 = 0, A = \{1\}, \Sigma = \{a, b\}$. $\delta(q,d)$ as shown in the transition table/diagram. This accepts strings that end in an odd number of a 's; e.g., $abbaaa$ is accepted, aa is rejected.

Input		
State	a	b
0	1	0
1	0	0

Transition Function/Table



Important Notes for Constructing the Finite Automata

For building the automata, first we start with the initial state. The FA will be in state k if k characters of the pattern have been matched. If the next text character is equal to the pattern character c , we have matched $k + 1$ characters and the FA enters state $k + 1$. If the next text character is not equal to the pattern character, then the FA goes to a state $0, 1, 2, \dots$ or k , depending on how many initial pattern characters match the text characters ending with c .

Matching Algorithm

Now, let us concentrate on the matching algorithm.

- For a given pattern $P[0..m-1]$, first we need to build a finite automaton F
 - The state set is $Q = \{0, 1, 2, \dots, m\}$
 - The start state is 0
 - The only accepting state is m
 - Time to build F can be large if Σ is large
- Scan the text string $T[0..n-1]$ to find all occurrences of the pattern $P[0..m-1]$

- String matching is efficient: $\Theta(n)$
 - Each character is examined exactly once
 - Constant time for each character
 - But the time to compute δ (transition function) is $O(m|\Sigma|)$. This is because δ has $O(m|\Sigma|)$ entries. If we assume $|\Sigma|$ is constant then the complexity becomes $O(m)$.

Algorithm:

```
//Input: Pattern string P[0..m-1],  $\delta$  and F
//Goal: All valid shifts displayed
FiniteAutomataStringMatcher(int P[], int m, F,  $\delta$ ) {
    q = 0;
    for (int i = 0; i < m; i++)
        q =  $\delta(q, T[i])$ ;
    if(q == m)
        printf("Pattern occurs with shift: %d", i-m);
}
```

Time Complexity: $O(m)$.

15.6 KMP Algorithm

As before, let us assume that T is the string to be searched and P is the pattern to be matched. This algorithm was presented by Knuth, Morris and Pratt. It takes $O(n)$ time complexity for searching a pattern. To get $O(n)$ time complexity, it avoids the comparisons with elements of T that were previously involved in comparison with some element of the pattern P .

The algorithm uses a table and in general we call it *prefix function* or *prefix table* or *fail function* F . First we will see how to fill this table and later how to search for a pattern using this table. The prefix function F for a pattern stores the knowledge about how the pattern matches against shifts of itself. This information can be used to avoid useless shifts of the pattern P . It means that this table can be used for avoiding backtracking on the string T .

Prefix Table

```

int F[]; //assume F is a global array
void Prefix-Table(int P[], int m) {
    int i=1,j=0, F[0]=0;
    while(i<m) {
        if(P[i]==P[j]) {
            F[i]=j+1;
            i++;
            j++;
        }
        else if(j>0)
            j=F[j-1];
        else {
            F[i]=0;
            i++;
        }
    }
}

```

As an example, assume that $P = a\ b\ a\ b\ a\ c\ a$. For this pattern, let us follow the step-by-step instructions for filling the prefix table F . Initially: $m = \text{length}[P] = 7, F[0] = 0$ and $F[1] = 0$.

Step 1: $i = 1, j = 0, F[1] = 0$

	0	1	2	3	4	5	6
P	a	b	a	b	a	c	a
F	0	0					

Step 2: $i = 2, j = 0, F[2] = 1$

	0	1	2	3	4	5	6
P	a	b	a	b	a	c	a
F	0	0	1				

Step 3: $i = 3, j = 1, F[3] = 2$

	0	1	2	3	4	5	6
P	a	b	a	b	a	c	a
F	0	0	1	2			

Step 4: $i = 4, j = 2, F[4] = 3$

	0	1	2	3	4	5	6
P	a	b	a	b	a	c	a
F	0	0	1	2	3		

Step 5: $i = 5, j = 3, F[5] = 1$

	0	1	2	3	4	5	6
P	a	b	a	b	a	c	a
F	0	0	1	2	3	0	

Step 6: $i = 6, j = 1, F[6] = 1$

	0	1	2	3	4	5	6
P	a	b	a	b	a	c	a
F	0	0	1	2	3	0	1

At this step the filling of the prefix table is complete.

Matching Algorithm

The KMP algorithm takes pattern P , string T and prefix function F as input, and finds a match of P in T .

```

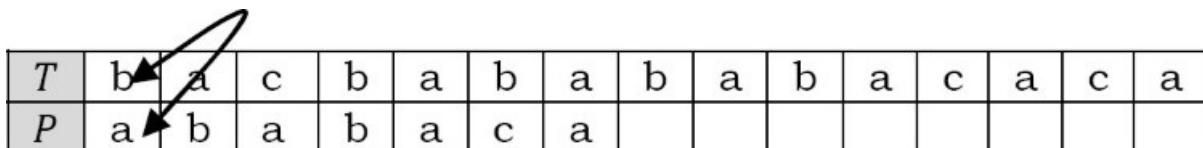
int KMP(char T[], int n, int P[], int m) {
    int i=0,j=0;
    Prefix-Table(P,m);
    while(i<n) {
        if(T[i]==P[j]) {
            if(j==m-1)
                return i-j;
            else {
                i++;
                j++;
            }
        }
        else if(j>0)
            j=F[j-1];
        else
            i++;
    }
    return -1;
}

```

Time Complexity: $O(m + n)$, where m is the length of the pattern and n is the length of the text to be searched. Space Complexity: $O(m)$.

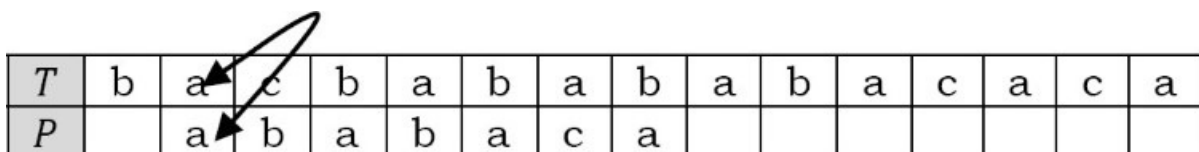
Now, to understand the process let us go through an example. Assume that $T = b a c b a b a b a b a c a c a$ & $P = a b a b a c a$. Since we have already filled the prefix table, let us use it and go to the matching algorithm. Initially: $n = \text{size of } T = 15$; $m = \text{size of } P = 7$.

Step 1: $i = 0, j = 0$, comparing $P[0]$ with $T[0]$. $P[0]$ does not match with $T[0]$. P will be shifted one position to the right.



T	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
P	a	b	a	b	a	c	a								

Step 2 : $i = 1, j = 0$, comparing $P[0]$ with $T[1]$. $P[0]$ matches with $T[1]$. Since there is a match, P is not shifted.



T	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
P		a	b	a	b	a	c	a							

Step 3: $i = 2, j = 1$, comparing $P[1]$ with $T[2]$. $P[1]$ does not match with $T[2]$. Backtracking on P ,

comparing $P[0]$ and $T[2]$.

T	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
P		a	b	a	b	a	c	a							

Step 4: $i = 3, j = 0$, comparing $P[0]$ with $T[3]$. $P[0]$ does not match with $T[3]$.

T	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
P				a	b	a	b	a	c	a					

Step 5: $i = 4, j = 0$, comparing $P[0]$ with $T[4]$. $P[0]$ matches with $T[4]$.

T	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
P					a	b	a	b	a	c	a				

Step 6: $i = 5, j = 1$, comparing $P[1]$ with $T[5]$. $P[1]$ matches with $T[5]$.

T	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
P					a	b	a	b	a	c	a				

Step 7: $i = 6, j = 2$, comparing $P[2]$ with $T[6]$. $P[2]$ matches with $T[6]$.

T	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
P					a	b	a	b	a	c	a				

Step 8: $i = 7, j = 3$, comparing $P[3]$ with $T[7]$. $P[3]$ matches with $T[7]$.

T	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
P					a	b	a	b	a	c	a				

Step 9: $i = 8, j = 4$, comparing $P[4]$ with $T[8]$. $P[4]$ matches with $T[8]$.

T	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
P					a	b	a	b	a	c	a				

Step 10: $i = 9, j = 5$, comparing $P[5]$ with $T[9]$. $P[5]$ does not match with $T[9]$. Backtracking on P , comparing $P[4]$ with $T[9]$ because after mismatch $= F[4] = 3$.



<i>T</i>	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
<i>P</i>					a	b	a	b	a	c	a				

Comparing $P[3]$ with $T[9]$.



<i>T</i>	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
<i>P</i>							a	b	a	b	a	c	a		

Step 11: $i = 10, j = 4$, comparing $P[4]$ with $T[10]$. $P[4]$ matches with $T[10]$.



<i>T</i>	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
<i>P</i>							a	b	a	b	a	c	a		

Step 12: $i = 11, j = 5$, comparing $P[5]$ with $T[11]$. $P[5]$ matches with $T[11]$.



<i>T</i>	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
<i>P</i>							a	b	a	b	a	c	a		

Step 13: $i = 12, j = 6$, comparing $P[6]$ with $T[12]$. $P[6]$ matches with $T[12]$.



<i>T</i>	b	a	c	b	a	b	a	b	a	b	a	c	a	c	a
<i>P</i>							a	b	a	b	a	c	a		

Pattern P has been found to completely occur in string T . The total number of shifts that took place for the match to be found are: $i - m = 13 - 7 = 6$ shifts.

Notes:

- KMP performs the comparisons from left to right
- KMP algorithm needs a preprocessing (prefix function) which takes $O(m)$ space and time complexity
- Searching takes $O(n + m)$ time complexity (does not depend on alphabet size)